🎤 AHAR — Demo Presentation Script

(Spoken Script for Interview / Hackathon Presentation)

> **How to use this:** Read it out loud as you demo the app. The `[ACTION]` tags tell you what to click or show on screen. The `...` markers are natural pause points.

---

1. Introduction *(~40 seconds)*

"Okay, so let me start with a quick real-life situation...

Imagine you're running a school canteen or a hospital cafeteria. Every single day, the kitchen manager has to make one really tough guess — '*How many meals should I cook today?*'

If they cook too little... people go hungry. If they cook too much... food gets thrown away. And that wasted food is not just a financial loss — it's also a huge environmental problem. Studies show that around one-third of all food produced in the world is wasted every year.

So I built **AHAR** — which stands for **AI-based Food Waste Prediction and Management System.**

AHAR helps kitchen managers make smarter decisions. It predicts how many meals to prepare, tracks ingredients in real time, logs what was cooked and eaten, shows waste trends on a dashboard, and even helps donate leftover food by finding nearby NGOs on a map.

It's basically... a smart assistant for your kitchen.

Let me walk you through how it works."

---

2. Live Demo Walkthrough

🔐 Step 1 — Logging In

`[ACTION: Open the app homepage and show the login/signup screen]`

"The first thing you see when you open the app is a login screen...

Now, building a secure login system from scratch — managing passwords, sessions, protecting user data — that's actually really complex and security-sensitive. So instead of building that from scratch, I used **Clerk**, which is a ready-made authentication service.

When I sign in here... Clerk handles the entire process — creating the account, verifying the email, managing the session token.

`[ACTION: Sign in and land on the Dashboard]`

And just like that, I'm logged in and taken to the main Dashboard."

📊 Step 2 — The Dashboard

`[ACTION: Show the Dashboard page with KPI cards and prediction history]`

"This is the main dashboard. It shows a summary of everything happening in the kitchen — prediction history, waste metrics, and quick stats.

Now here's something interesting about what you see here...

All this prediction history is loaded from the **database** — not from the browser. In an earlier version, I was storing predictions in the browser's local storage, which meant if the server restarted, all the history would disappear. That wasn't good enough.

So I added a **MongoDB** collection called `PredictionLog`. Every time a prediction is made, it gets saved there permanently. So even after a server restart, the dashboard still shows the full history.

The frontend — which is built using **React** — fetches this data when the page loads and displays it dynamically. React is great for this because whenever the data changes, it automatically updates only the part of the screen that changed — you don't need to refresh the whole page."

🧠 Step 3 — Demand Prediction *(Core Feature)*

`[ACTION: Navigate to the Prediction page]`

"Now let me show you the most important feature — **Demand Prediction.**

This is where the kitchen manager comes every morning before cooking starts.

`[ACTION: Fill in the form — past consumption numbers, expected people, day of week, weather, events]`

So I'm entering... the past few days' consumption — say 120, 130, 115 — the expected number of people today — let's say 145 — it's a Friday, the weather is rainy, and there's a special event happening — a 'Founders Day'.

Now I'll click Predict...

`[ACTION: Click the Predict button and show the result]`

And here's the result — it's recommending **178 meals** today. It also says there's a **surplus risk** and even **recommends donating**.

Now let me explain what just happened behind the scenes — because this is the interesting part.

When I clicked Predict, the **React** frontend sent a request to my **Node.js** backend with all this data in JSON format.

The backend then:

First — it went to the **MongoDB** database and looked up if 'Founders Day' is a known event. It found a record saying this event increases demand by 30%. So it applied a **1.3 multiplier**.

Second — it noticed the weather is 'Rainy'. Rainy days tend to increase meal demand slightly — people prefer hot food — so it applied a **1.05 multiplier**.

Third — it calculated the base demand using a weighted formula: 70% weight on past history, 30% on expected headcount. The idea being — past data is usually more reliable than someone's guess about attendance.

So the final prediction is: `base × 1.3 × 1.05 = 178 meals.`

Then... it saved this prediction to the database so the dashboard can show it later.

And all of that happened in under half a second.

`[ACTION: Point to the 'ml' field in the response showing provider: 'fallback' or 'ml-service']`

You'll also notice it tells you whether the prediction came from the **ML service** or a **fallback formula**. I'll explain that in a minute."

📦 Step 4 — Inventory Management

`[ACTION: Navigate to the Inventory page]`

"Next is the **Inventory page**. This is where all ingredients are tracked.

`[ACTION: Show the ingredient list — rice, oil, salt with stock quantities]`

You can see every ingredient — name, stock quantity, unit, and a reorder indicator.

Now let me add a new ingredient...

`[ACTION: Fill in and submit the form to add 'Tomatoes — 20 kg']`

Done. That was a POST request from React to my Node.js API, which saved it to **MongoDB**.

But here's a smart thing I built in — if you try to add 'Rice' again when it already exists in this kitchen... the system doesn't create a duplicate. It just **adds the stock quantity** to the existing record. So you never have two separate 'Rice' entries.

This is because I gave MongoDB a **unique index** on the combination of kitchen ID and ingredient name. If a duplicate comes in, MongoDB throws an error code `11000`, and my backend catches it and handles it gracefully...

`[ACTION: Show the requirements calculator tab]`

And this tab is really useful — you enter the predicted meal count and select which dishes you're cooking today. It instantly shows you **exactly how much of each ingredient is needed** and flags any shortages. This saves the manager from manually doing these calculations every morning."

🍽️ Step 5 — Menu Management

`[ACTION: Navigate to Menu Management]`

"Here's where dishes are defined. Each dish has a list of ingredients with exact amounts per meal.

`[ACTION: Show a dish like 'Chicken Fried Rice' with ingredients: Rice 0.2kg, Oil 0.05kg]`

This matters because when a meal is logged later, the system uses these ingredient amounts to automatically deduct stock. So the kitchen manager doesn't have to manually track 'I used 24 kg of rice today' — the app calculates it for them."

📝 Step 6 — Consumption Logging

`[ACTION: Navigate to Consumption Form]`

"Every day after the meal service, the manager logs what was actually cooked and consumed.

`[ACTION: Fill in — Dish: Chicken Fried Rice, Cooked: 120, Consumed: 105]`

So 120 meals were cooked, 105 were eaten... that means 15 meals were wasted. I'll submit this.

`[ACTION: Submit and show success message]`

Behind the scenes, a few things happen simultaneously:

One — the **ConsumptionLog** is saved to MongoDB with cooked: 120, consumed: 105, leftover: 15.

Two — the system fetches the ingredients for Chicken Fried Rice from the Dish model... calculates total rice used: 0.2 kg × 120 = 24 kg... and then **decrements the inventory** by 24 kg.

The decrement uses a special MongoDB operation called `$inc` which does the subtraction atomically — meaning even if two people submit at the exact same time, the stock update is always correct. No data gets messed up."

📈 Step 7 — Analytics Dashboard

`[ACTION: Navigate to Analytics page]`

"Now let's look at the analytics.

`[ACTION: Show the charts — daily waste totals, weekly trends, dish-wise waste breakdown]`

These charts show daily waste, weekly trends, and which dishes waste the most.

Something important to highlight here — this data is not processed in the app code. I'm using **MongoDB Aggregations** to compute all of this directly inside the database.

Think of it like running an Excel formula inside the database server itself. Only the final summary travels to my app — not thousands of raw records. This keeps the app fast even as data grows.

For example — the 'dish-wise waste' chart uses a MongoDB $group to add up waste per dish, and a $lookup to get the dish name. It's the database doing the heavy lifting."

🗺️ Step 8 — Donation Locator

`[ACTION: Navigate to Donation Locator page]`

"When the prediction flags a surplus risk, the app helps find NGOs nearby to donate the food.

`[ACTION: Allow location access and show the map with NGO pins]`

The map is powered by **Google Maps JavaScript API**. The NGO search uses a free service called **Overpass API** — which is basically a search engine for map data. It finds food banks and charities within a few kilometers.

I also store NGOs in MongoDB with a special **geo index** — so if Overpass is down or too slow, the app falls back to our own cached NGO list. The user always sees results, no matter what."

🤖 Step 9 — ML Waste Prediction & Dish Recommendations

`[ACTION: Show the waste prediction form or recommendations page]`

"Here's where the **actual machine learning** comes in.

I have a separate Python service running in the background — built with **FastAPI**. It loads a trained **scikit-learn** model from disk, and exposes it through an HTTP endpoint called `/predict`.

When the Node.js backend needs a waste prediction, it sends the kitchen's features — occupancy rate, temperature, past meal data, weather type, menu type — to this Python service... the model analyzes them and returns a number.

`[ACTION: Send a waste prediction request and show the ML response]`

See here — the response shows `provider: ml-service` meaning the real ML model answered.

But here's the important part — what if the ML service is down? Say the Python server crashed overnight...

`[ACTION: Show or describe the fallback behavior]`

My Node.js backend has a **timeout of 5 seconds** on every ML call. If the ML service doesn't reply in time, the backend automatically switches to a fallback formula — a simple math calculation based on past averages. The user still gets a prediction. They don't even know the ML was unavailable.

That kind of resilience is very important in production systems.

The dish recommender works similarly — the ML service ranks candidate dishes based on cuisine preferences and prep time, and if it's unavailable, the backend returns the first few dishes from the menu as a simple

fallback."

---

3. Multitask Handling — How It Handles Many Users

"Let me explain something that's actually a really common interview question — '*What happens when 1000 users use your app at the same time?*'

Imagine a busy restaurant waiter. If the waiter is old-school... they take one order, walk to the kitchen, stand there and wait for the food to be ready, bring it back, and only THEN take the next customer's order. That would be incredibly slow.

But a smart waiter does this instead: take order from Table 1... go to the kitchen and place the order... while the kitchen is cooking... go take Table 2's order... go place that order too... come back when Table 1's food is ready and serve it.

The waiter is never standing idle. They're always doing something useful while waiting.

That's exactly how **Node.js** works. It runs on what's called an **event loop** — a single smart worker that never waits idle. When a user request comes in and needs to wait for MongoDB to respond... Node.js doesn't freeze. It picks up the next request and handles that while waiting. When MongoDB responds, it comes back and continues.

This is called **asynchronous** or **non-blocking** behavior. It's why Node.js can handle thousands of requests simultaneously even though it's running on a single thread.

So all the `await` keywords you see in my code — like `await ConsumptionLog.create(...)` — those are the moments where Node.js hands control back to the event loop while waiting for MongoDB.

Now, what about really heavy traffic — like 10,000 concurrent users?

For that, you'd run multiple copies of the Node.js backend behind a **load balancer**. The load balancer is like... a traffic police officer at a busy intersection. It looks at all the backend servers and sends each new request to the one that's least busy.

To protect the system from crashing:

- Input validation runs first — bad requests are rejected immediately before any DB work
- The ML service has a 5-second timeout with a fallback, so it never blocks indefinitely
- MongoDB is connected with a retry mechanism, so a brief DB hiccup doesn't crash the whole server"

---

4. Tech Deep Dive — Why This Stack?

"Let me quickly explain why I chose each technology — because interviewers always ask this.

Why MongoDB? I chose MongoDB because it's a document database — it stores data like JSON, which fits perfectly with JavaScript. More importantly, during development I was changing the data structure often. With MongoDB, adding a new field is as simple as just starting to use it. In a traditional SQL database, every change

requires a formal 'migration' — adding or altering a column in the table. That slows down fast-paced development significantly.

Also, Dishes naturally need to include a list of ingredients inside them. In MongoDB, I can embed that list directly in the Dish document. In SQL, I'd need a separate join table and an extra query every time I want to read a dish with its ingredients.

Why Node.js? My backend is mostly doing I/O work — waiting for MongoDB queries, waiting for the ML service to respond. It's not doing heavy CPU calculations. Node.js excels at I/O-heavy work because of its async event loop. It handles many simultaneous requests without creating a separate thread for each one, which saves a lot of memory.

Why React? A kitchen management app has multiple pages — Inventory, Analytics, Predictions, Menus — and lots of interactive forms and charts. React is ideal for this because it lets you build these as independent components that manage their own state. When data changes, only the affected part of the screen updates. This makes the UI fast and responsive without full page reloads.

Why a separate ML service in Python? Node.js can't directly run Python machine learning libraries like scikit-learn or pandas. Instead of fighting that limitation, I built the ML model as a completely separate Python service using FastAPI. The Node.js backend calls it like any other web API — sends a request, gets a response. This also means I can upgrade the ML model without touching the backend at all. And if needed, I can deploy the ML service on a more powerful machine with more CPU or even a GPU.

The main trade-off? Running three separate services — frontend, backend, and ML — adds deployment complexity. You have to manage three processes and ensure they can reach each other. But the benefit of clean separation and independent scaling outweighs that cost for a production system."

---

5. Challenges & Solutions

"Let me be honest about the real challenges I faced...

Challenge 1: ML library version mismatch I trained the scikit-learn model on one machine and deployed it on another with a slightly different version of scikit-learn. The preprocessing pipeline inside the saved model file would crash. I solved this by removing the dependency on the saved pipeline entirely and manually encoding all the input features in Python code. That way, the code doesn't care which version of scikit-learn is installed — it always prepares the data correctly.

Challenge 2: Prediction history disappearing after restart Originally I was storing prediction history in the browser's localStorage. Every time the server restarted during development, all that history was gone. I created a `PredictionLog` MongoDB collection and started saving every prediction there. Now the history is permanent and loads from the database on every page visit.

Challenge 3: External API failures The Overpass API — which I use for finding nearby NGOs — sometimes returns errors or is slow. Without any fallback, the Donation feature would just fail. I added a try-catch around the Overpass call: if it fails, the backend returns NGO records from our own MongoDB instead. The feature keeps working even when the external API is down.

Challenge 4: Inventory going out of sync When logging consumption, I need to both save the consumption record AND reduce the ingredient stock. These are two separate database operations. If the second one fails, the inventory is wrong. The fix was using MongoDB's atomic `$inc` operator for stock updates and organizing the code so errors in stock update are caught and logged without crashing the entire consumption save."

---

6. Future Improvements

"If I were to take this to production, here's what I'd add:

Scaling: I'd run multiple backend instances behind a load balancer like Nginx. MongoDB would use a replica set — which is basically multiple copies of the database — so reads are spread across them and there's no single point of failure.

Caching: The Analytics dashboard runs heavy aggregations every time it loads. I'd add a Redis cache — think of it as a fast short-term memory — so the aggregation result is stored for 10–15 minutes. Instead of hitting MongoDB on every load, most users get the cached result instantly.

Background jobs: Instead of saving every prediction to MongoDB synchronously during the API call, I'd use a job queue. The prediction response goes back to the user immediately, and the DB save happens in the background. This makes the API faster.

Better ML model: The current model is a linear regression trained on limited data. With more real-world kitchen data — across different facility types, seasons, and meal patterns — I'd move to a gradient boosting model like XGBoost which would give significantly better accuracy.

Real backend auth on ML endpoints: Right now, Clerk authentication is enforced mainly on the frontend. I'd add JWT token verification on every backend API route so the server independently checks that the request is from a logged-in user — not just trusting the frontend."

---

7. Closing

"So to summarize — AHAR is a full-stack, AI-powered kitchen management system built with React on the frontend, Node.js and Express on the backend, MongoDB as the database, and a Python FastAPI service for machine learning.

It solves a real problem — food waste in institutional kitchens — and it's designed with real production concerns in mind: graceful fallbacks when services fail, atomic database operations to prevent inconsistency, and a modular architecture that's easy to scale.

The project taught me how to design a system where the parts work together smoothly — and more importantly, how to handle things gracefully when they don't.

I'm happy to go deeper on any part of this — the ML model, the database design, the async behavior, or the scaling strategy."

> 💡 **Tip:** Before your demo, make sure all three services are running:

> - Frontend: `npm run dev` inside `code/frontend/`
> - Backend: `npm start` inside `code/backend/`
> - ML Service: `uvicorn app:app --reload` inside `code/ml-service/`